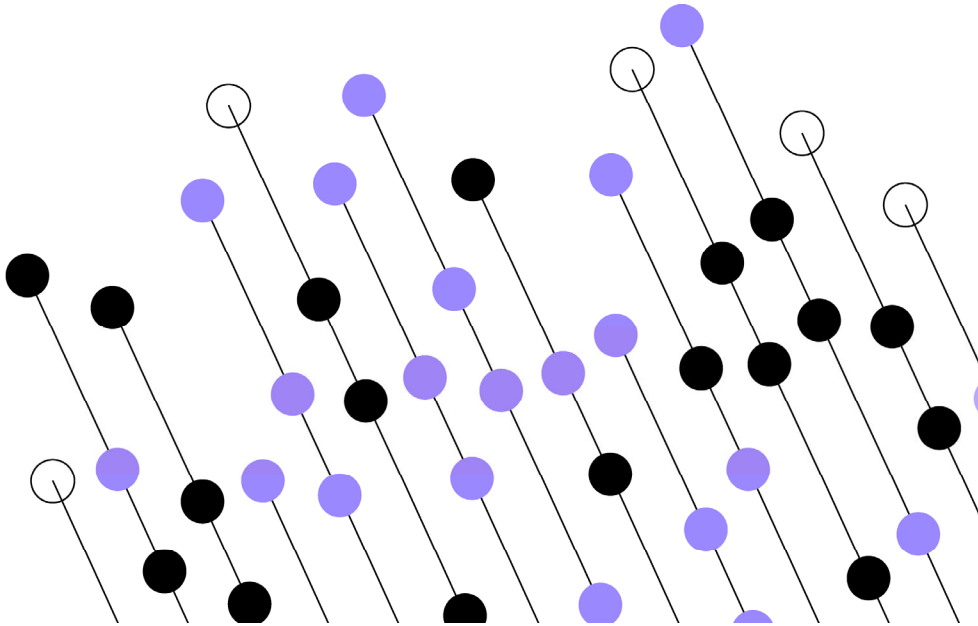
Methods



Example of a method with no parameter and no returned value

```
public class Program {
    public static void main(String[] args) {
        updateAllSalaries();
    }

    private static void updateAllSalaries() {
        // Code to update all salaries
    }
}
```

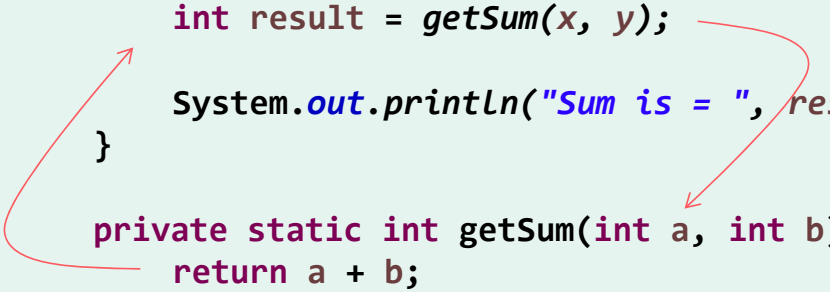
The caller just calls the method and lets it do its work!

Example of a void method with two parameters

```
public class Program {
    public static void main(String[] args) {
        int x = 1, y = 2;
        add(x, y);

        add(3, 7);
    }

    private static void add(int a, int b) {
         System.out.println(a + b);
    }
}
```

Example of method returning a value

```
public class Program {
    public static void main(String[] args) {
        int x = 1, y = 2;

        int result = getSum(x, y);

        System.out.println("Sum is = ", result);
    }

    private static int getSum(int a, int b) {
        return a + b;
    }
}
```



A method can only return one thing.
The caller decides what to do with the returned value.

Method overloading

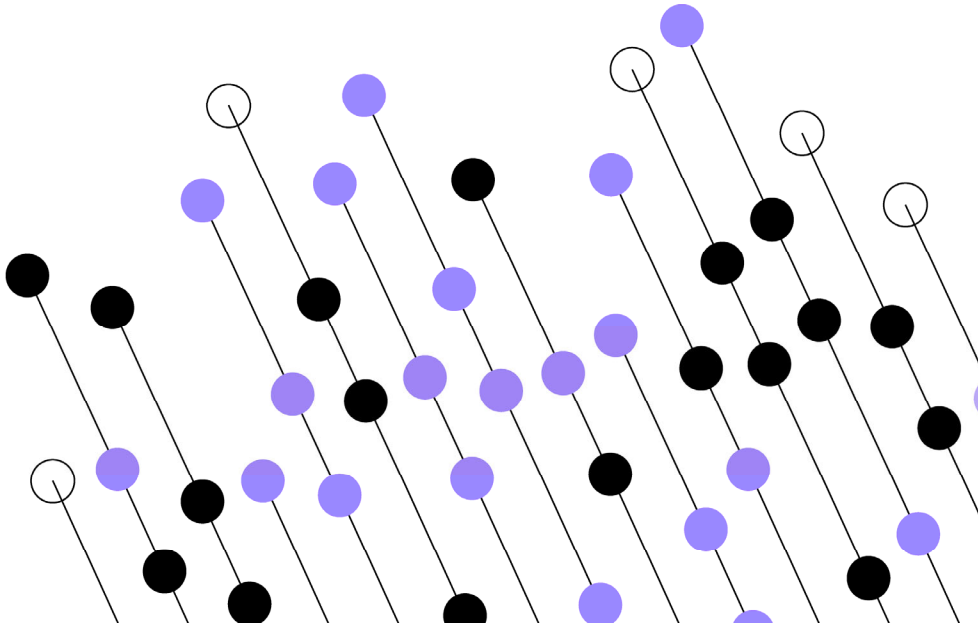
```
public class Program {
    public static void main(String[] args) {

        double result1 = getTax(2000);
        System.out.println(result1);

        double result2 = getTax(2000, 0.4);
        System.out.println(result2);
    }

    private static double getTax(double salary) {
         return salary * 0.25;
    }

    private static double getTax(double salary, double rate) {
        return salary * rate;
    }
}
```

Prints **500**

Prints **800**

Same name
different parameters.

A few Java library methods



Introducing some Java library methods

Use the `java.util.Scanner` class to input a value.

```
Scanner s = new Scanner(System.in);

System.out.println("What is your name?");
String name = s.nextLine();

System.out.println("What is your age?");
int age = s.nextInt();

System.out.println(    "Hi " + name + "\n next year you'll be " + (age + 1));
```

Printing formatted output

- `System.out.println(..)` is hugely overloaded.
- Can build a 'String' via concatenation (using +). This is onerous and error prone (spacing).

```
int age = 21;
String name = "Bob";
System.out.println("Hi " + name + ", next year you will be " + age + 1);
```

Hi Bob, next year you will be 211.

- Best use **printf**() and **String.format**() for formatting.

```
System.out.printf("Hi %s, next year you will be %d", name, age + 1);
```

Format string

Examples — printf()

%s used to represent a String.

```
String word = "wibble";
System.out.printf("My favourite word is %s\n", word);
```

%d	for an int value
%8d	len 8, right justified
%08d	with leading zeros
%+8d	include sign +/-
%,8d	thousand separator

```
int num = 123456;
System.out.printf("%d\n",    num);   "123456"
System.out.printf("%08d\n",  num);   "00123456"
System.out.printf("%+8d\n",  num);   " +123456"
System.out.printf("%,8d\n",  num);   " 123,456"
System.out.printf("%+,8d\n", num);   "+123,456"
```

String.format is at the heart of formatting any string

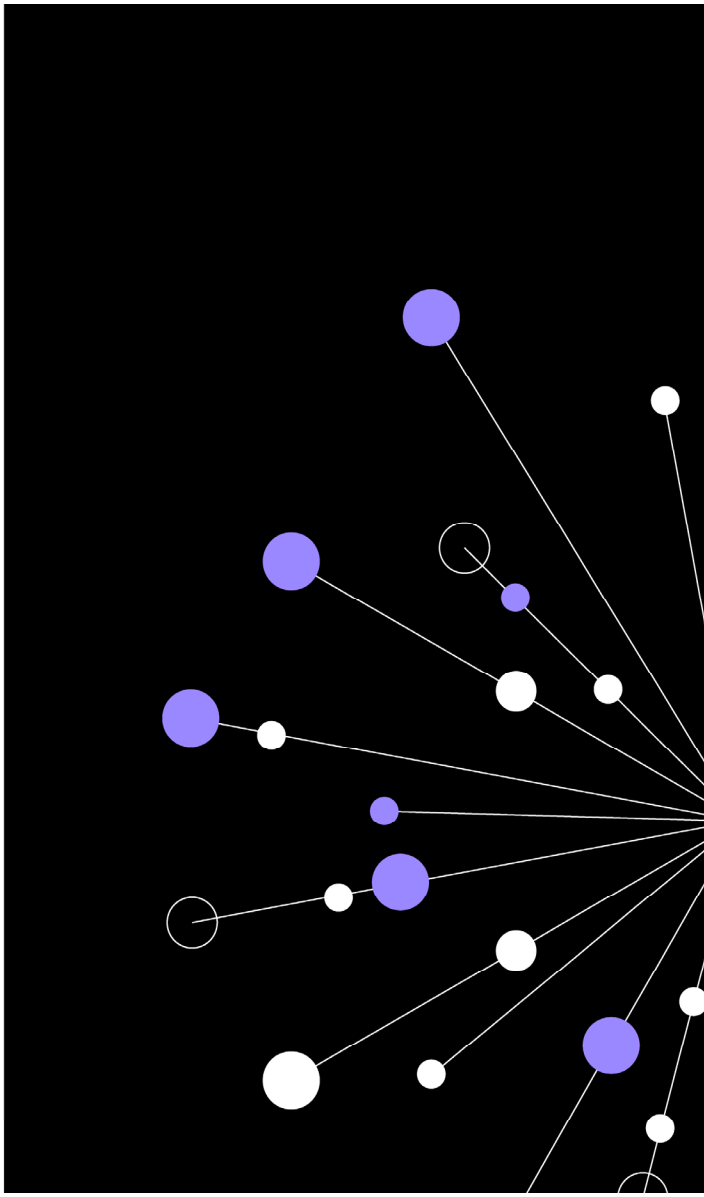
```
String name = "Bob";
int age = 28;
double balance = 12345.67;


String message = String.format(
        "Hello %s! You are %d years old and your balance is $%,.2f.",
        name, age, balance
);

System.out.println(message);
```

Lab

Objectives:

- Practice declaring and using methods.
- Implement overloading.
- Use Java library methods to get data from users.

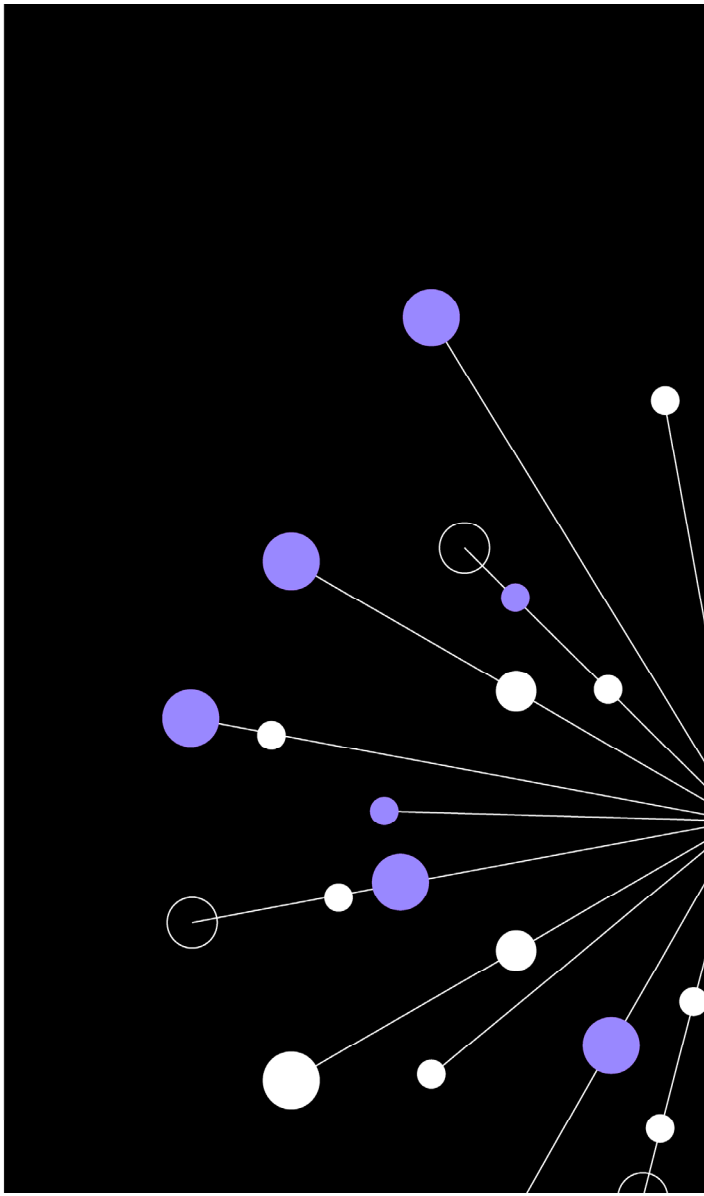
Duration: 90 minutes



QA

In this chapter, we reviewed...

- Creating Methods
- Java library methods



QA